

React Phase 1 — Interview Notes

Everything here was implemented, not just read: `UserCard` (props → state → lifted state) and the Expense Tracker (form, list, total). That distinction matters in interviews — you can talk about the bug you hit, not just recite the concept.

1. The core mental model

A React component is **a function that takes data and returns a description of UI**. React calls your function, compares the returned output to what's on screen, and patches only the differences.

Two consequences worth stating explicitly:

- **Your component function runs many times** — on every render. Never do work inside it that shouldn't repeat.
- **UI is a function of state**. You never reach into the DOM to change things. You change state; React re-derives the screen.

Interview phrasing: "React is declarative — I describe what the UI should look like for a given state, rather than writing imperative instructions to mutate the DOM."

2. JSX

JSX compiles to function calls: `<Card title="Hi" />` becomes `Card({ title: "Hi" })`.

That single fact explains every rule:

Rule	Why
One root element	A function returns one thing. Use <code><>...</></code> (Fragment) to group without adding a DOM node.
<code>className</code> not <code>class</code>	<code>class</code> is a reserved JavaScript keyword.
<code>{ }</code> escapes into JS	Anything inside braces is a normal JavaScript expression.

Capitalized component names	Lowercase compiles to an HTML tag string; capitalized resolves to your component variable.
--------------------------------	---

3. Props

Data passed **down** from parent to child.

- **Read-only.** A component can never modify its own props. The data belongs to the parent.
- **Changing props re-renders the child.** New props in → React re-runs the function → DOM updates.

Functions passed as props are how children trigger changes in parents. Used twice in Phase 1: `onToggle` in `UserCard`, `onAddExpense` in `ExpenseForm`. The child never touches parent state directly — it calls the function it was handed. This is *the* pattern for child → parent communication.

```
type ExpenseFormProps = {  
  onAddExpense: (expense: Expense) => void; // takes an Expense, returns nothing  
};
```

4. State — `useState`

Data a component **owns and can change**; changing it triggers a re-render.

```
const [expenses, setExpenses] = useState<Expense[]>([]);
```

- `expenses` — current value, read like a variable.
- `setExpenses` — the *only* correct way to change it.
- The argument is the **initial** value, used only on the first render.
- `<Expense[]>` — the generic. Required when the initial value is empty (`[]` or `{}`), because TypeScript can't infer contents from emptiness. Without it, `[]` infers as `never[]` and every later `setExpenses` errors.

Three things interviewers probe

(a) Updates are asynchronous and batched. `setX(...)` doesn't change `x` in the current function scope — it schedules a re-render, and the new value appears on the *next* render.

(b) Use the updater form when new state depends on old state:

```
setExpenses(prev => [...prev, expense]); // correct — prev is always latest
setExpenses([...expenses, expense]);      // works, but can read stale state
```

The direct form closes over `expenses` from the current render, which may be stale if updates are batched.

(c) Immutability is required. React decides whether to re-render by comparing references.

`expenses.push(x)` mutates the same array — same reference — so React sees no change and skips the re-render. `[...prev, x]` creates a new array with a new reference, so React detects it. Same reasoning behind `{...prev, [id]: !prev[id]}` for objects.

5. Conditional rendering

```
{isOnline ? <span className="green-circle" /> : <span className="red-circle" />}
{expenses.length === 0 ? <p>No Expenses Added</p> : <ul>...</ul>}
```

Ternary for either/or, `&&` for show/hide.

`&&` gotcha: `{count && <p>...</p>}` renders a literal `0` when count is 0 — `0` is falsy but still a valid React child. Use `{count > 0 && ...}`.

6. Lists and `key`

```
{expenses.map(expense => (
  <li key={expense.id}>{expense.name}</li>
))}
```

Why `key` exists: React needs stable identity per item to know which one changed, moved, or was removed between renders, so it can patch surgically instead of rebuilding.

Why not the array index: if items reorder or get deleted, indices shift and React misattributes state to the wrong item — the classic bug where you delete row 2 and row 3's input value appears in its place. Use a stable unique `id`. `crypto.randomUUID()` at creation time solves this.

7. Lifting state up

Problem: state inside a child is invisible to its parent and siblings. When `isOnline` lived inside each `UserCard`, `App` couldn't count how many users were online.

Solution: move state to the nearest common ancestor; pass the value down as a prop and a callback down to change it.

Applied twice:

- **UserCard** — state moved to `App`; `isOnline` + `onToggle` passed down.
- **Expense Tracker** — `expenses` lives in `App`; `ExpenseForm` writes via `onAddExpense`, `ExpenseList` and `TotalExpense` read. One source of truth.

The resulting architecture is the point: children became **presentational** — pure functions of their props, no internal state, easy to test and reuse. The parent owns data and coordination. This container/presentational split is standard React architecture and worth naming in an interview.

8. Derived state — the junior/mid-level dividing line

Never store in state something you can calculate from existing state.

```
// correct — computed during render
const total = expenses.reduce((sum, e) => sum + e.amount, 0);

// wrong — redundant state that can drift out of sync
const [total, setTotal] = useState(0);
useEffect(() => { setTotal(expenses.reduce(...)) }, [expenses]);
```

The second costs an extra render, adds maintenance, and introduces a real bug class: the total going stale relative to the list.

Where to compute it: as close as possible to where it's consumed. `reduce` lives inside `TotalExpense`, not `App` — one consumer, logic co-located with its use. Lift the calculation up only when multiple siblings need the same derived value.

9. Controlled inputs

An input is **controlled** when React state is the source of truth for its value.

```
<input value={name} onChange={e => setName(e.target.value)} />
```

What happens per keystroke: you type → React's `value` prop still holds old state, so React overrides the display → `onChange` fires with `e.target.value` → `setName` updates state → re-render → `value` shows the new character. Looks like normal typing, but state drives the DOM every character.

Why bother: enables live transformation (force uppercase, reject non-digits), validation as you type, and programmatic reset (`setName("")` after submit — no DOM refs). The value lives in exactly one place instead of split between the DOM and your component.

Uncontrolled is the alternative — DOM owns the value, read via a ref. Occasionally simpler; loses all the above. Default to controlled.

`e.preventDefault()` **is mandatory** in submit handlers. Without it the browser's default form behavior reloads the page and wipes all in-memory React state.

10. TypeScript patterns used

Pattern	Example	Purpose
Shared type file	<code>types.ts</code> exporting <code>Expense</code>	Single source of truth for a data shape
Props type	<code>type ExpenseListProps = { expenses: Expense[] }</code>	Contract for what a component accepts
Function prop type	<code>onAddExpense: (expense: Expense) => void</code>	Types a callback prop
<code>useState</code> generic	<code>useState<Expense[]>([])</code>	Required when initial value is empty
Event types	<code>React.ChangeEvent<HTMLInputElement> ,</code> <code>React.FormEvent<HTMLFormElement> ,</code> <code>React.ChangeEvent<HTMLSelectElement></code>	Correctly typed DOM events, matched to element
Type-only		

```
import
```

```
import type { Expense } from "../types"
```

```
Erased at build time
```

Honest limitation worth knowing: `Number("abc")` is `NaN`, which is still type `number` — TypeScript won't flag it. Types catch shape errors, not value errors; runtime validation is still required. Good answer to *"what can't TypeScript do for you?"*

11. Bugs actually hit — talk about these

"Too many re-renders" — from `onClick={setIsOnline(!isOnline)}` without the arrow wrapper. The function *calls* immediately during render → sets state → re-renders → calls again → infinite loop. `onClick={() => setIsOnline(!isOnline)}` passes a function *reference* that only runs on click. Explaining this from experience beats reciting it.

undefined VS false — an unvisited key in `onlineStatus` returns `undefined`, not `false`. `!!value` coerces to a real boolean, which the `isOnline: boolean` prop type requires. Shows you understand type boundaries.

12. Likely interview questions on Phase 1

1. Why can't you mutate state directly? (*reference comparison* → *no re-render*)
 2. What does the `key` prop do, and why not use the index?
 3. Difference between props and state?
 4. `setCount(count + 1)` VS `setCount(prev => prev + 1)` — when does it matter?
 5. Controlled vs uncontrolled inputs — pros and cons?
 6. Why does `e.preventDefault()` matter in a React form?
 7. How do you share state between two sibling components?
 8. How does a child update its parent's state?
 9. When should you store something in state vs. compute during render?
 10. What triggers a re-render in React?
-
-

Appendix: React 18 vs React 19 — what interviewers actually ask

You're learning on React 19, which is correct. But most production codebases you'll interview for are still on 18, and interviewers ask about 18 features constantly. **Nothing in Phase 1 differs between the versions** — props, state, JSX, keys, lifting state, controlled inputs are identical in both. The differences start at the concurrent/async features, which is Phase 2 onward.

React 18 — the concurrency release (2022)

`createRoot` replaced `ReactDOM.render`

```
// 17
ReactDOM.render(<App />, document.getElementById('root'));
// 18+
ReactDOM.createRoot(document.getElementById('root')).render(<App />);
```

This opt-in is what enabled everything else below. Your `main.tsx` already uses

`createRoot`.

Automatic batching — in 17, multiple `setState` calls were batched only inside React event handlers. In 18, they're batched everywhere: promises, `setTimeout`, native event handlers. Two `setState` calls in a `fetch().then()` now cause one re-render instead of two. *Common interview question.*

Concurrent rendering — React can now interrupt, pause, and resume a render instead of blocking the main thread until done. Not a feature you call directly; it's the engine behind the hooks below.

`useTransition` — mark a state update as low-priority so urgent updates (typing) don't get blocked by expensive ones (filtering a huge list).

```
const [isPending, startTransition] = useTransition();
startTransition(() => setSearchResults(expensiveFilter(query)));
```

`useDeferredValue` — similar goal, different shape: pass it a value and get back a possibly-stale version that lags behind during heavy renders.

`useId` — generates stable unique IDs that match between server and client, for accessibility attributes (`htmlFor` / `id` pairs).

`useSyncExternalStore` — for subscribing to external stores safely under concurrent rendering. Mostly used by library authors (Zustand, Redux internals).

StrictMode double-invoking — in development, React 18 mounts, unmounts, then remounts components, and runs effects twice. This is intentional: it surfaces missing `useEffect` cleanup functions. It does *not* happen in production. **This trips up every beginner** — when you get to Phase 2 and see your effect fire twice, this is why.

Suspense for SSR + streaming — server-rendered HTML can stream in pieces rather than waiting for everything.

React 19 — the actions release (late 2024 / 2025)

Actions & `useActionState` — built-in handling of pending/error/success states for async form submissions, replacing a lot of manual `useState` boilerplate.

```
const [state, formAction, isPending] = useActionState(submitFn, initialState);
<form action={formAction}>...</form>
```

`useOptimistic` — show the expected result immediately while the request is in flight, roll back automatically on failure.

`useFormStatus` — lets a nested component (e.g. a submit button) read the parent form's pending state without prop drilling.

`use()` — read a promise or context during render; unlike hooks, it can be called conditionally.

ref as a regular prop — `forwardRef` is no longer needed for function components. If you see `forwardRef` in a codebase, it's 18-era code.

<Context> as provider — `<ThemeContext value={x}>` instead of `<ThemeContext.Provider value={x}>`.

Document metadata — `<title>`, `<meta>`, `<link>` rendered anywhere in a component tree get hoisted into `<head>` automatically. No more react-helmet.

Better hydration error messages and improved error reporting generally.

React Compiler (shipped stable alongside the 19 line) — auto-memoizes at build time, so hand-written `useMemo` / `useCallback` / `React.memo` become largely unnecessary. **You still need to know what they do** — they're all over existing codebases and interview questions. This is Phase 5 material.

How to frame this in an interview

If asked what version you know: *"I've been building on React 19, but I'm comfortable with the 18 codebase patterns too — `forwardRef`, `Context.Provider`, manual memoization, manual form pending states — since most production apps are still there."*

That answer signals current knowledge **and** practical readiness for their actual codebase, which is what they're really screening for. Claiming only-latest knowledge can read as inexperience with real systems.

Most likely version questions:

1. What changed in React 18 with automatic batching?
 2. Why does StrictMode run effects twice in development?
 3. What problem does `useTransition` solve?
 4. What replaced `forwardRef` in React 19?
 5. What does the React Compiler do, and does it mean `useMemo` is dead?
-

Next: Phase 2

`useEffect` and its cleanup, the dependency array, "you might not need an effect", custom hooks, data fetching with loading/error/empty states, `AbortController` and race conditions. Project: Weather/Movie Search Dashboard.